

CS151 Intro to Data Structures

Java Review: Classes, User Input, and Exceptions

Announcements

- Join Piazza
- Join Gradescope
 - **8YG6VP**
- TA Office Hours 5-7pm
 - **Monday:** Khanh Ha
 - **Tuesday:** Maimoona
 - **Thursday:** Ellen
 - **Friday:** Huda
- Prof Office Hours
 - Take the poll on piazza
- HW0 released
 - due Friday Sep 11
 - Start early!

Outline

- Review: College Exercise from last class
- Exceptions and I/O
- OOP and Inheritance

Exercise 1 -

Part a: Create a College class with:

name,

number of students,

year founded

Part b: In the main, create 3 colleges and put them in an array

Part c: Take a college name as input and print the year it was founded

User Input

There are two ways to read in data that we will use in this course. The major difference is when the data is supplied.

For command line arguments the data is supplied directly when you run the program.

For example: `java Add 5 7`

With the **Scanner**, we can take input while the program is running. For example:

```
$ java Add
Please supply which numbers you would like to add
5
7
```

User Input

```
Scanner input = new Scanner(System.in);

System.out.print("Enter your name: ");
String name = input.nextLine();    // reads a whole line

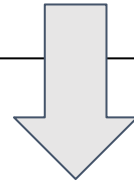
System.out.print("Enter your age: ");
int age = input.nextInt();         // reads a single int

System.out.print("Enter your GPA: ");
double gpa = input.nextDouble();  // reads a single double
```

Exercise 1

Modify the College program to take user input during runtime (Scanner) rather than as a command line argument

User Input - Reading from a file



```
Scanner input = new Scanner(System.in);
```


Exercise 2 - count words in a file

Part a: Read in a filename from command line

Part b: Count the number of words in the file

Exceptions – way to deal with unexpected events during execution

- Unexpected events:
 - unavailable resource
 - unexpected input
 - NPE
 - AOB

How do we deal with exceptions?

```
try {  
    guardedBody  
} catch (exceptionType1 variable1) {  
    remedyBody1  
} catch (exceptionType2 variable2) {  
    remedyBody2  
} ...  
...
```

Exceptions

code :)

Exceptions

1. Checked Exceptions

- a. ``error: unreported exception FileNotFoundException;
must be caught or declared to be thrown'`

2. Unchecked Exceptions

- b. `ArrayIndexOutOfBoundsException`
- c. `NullPointerException`
- d. `ArithmeticException`

Exceptions

How do we deal with them?

- a) in the caller

- a) in the callee

Exceptions

- Exceptions are objects
 - can create with `new` keyword
- Inheritance
 - `NullPointerException` **is a** `RuntimeException` **is an** `Exception`
 - `FileNotFoundException` **is a** `IOException` **is an** `Exception`

Object Oriented Programming

Software Design Goals

- Robustness
 - software capable of error handling and recovery
- Adaptability
 - software able to evolve over time and changing conditions (without huge rewrites)
- Reusability
 - same code is usable as component of different systems in various applications

Object Oriented Programming aims to achieve these!

What benefits does a Class give us?

Abstraction and Encapsulation - hide internal details of how an Object works, while providing a well defined way to interact with it

Access Modifiers

- public - accessible from anywhere. Any other class can read, call, or use it.
- private - accessible only inside the same class.

BankAccount.java and Main.java

Inheritance

- Enables a class to use the properties and behaviors of another class
- Establishes relationships between classes

Towards our goal of reusability!

Inheritance

Student example code

super

- `super` refers to the superclass object
- can also be used to reference methods defined in the superclass
 - `super ( . . . . . )` references the parent class constructor
- `super.getName ( )`

Inheritance - constructors

- Constructors are never inherited
- A subclass may invoke the superclass constructor via a call to `super` with the appropriate parameters
- If calling `super`, it must be in the first line of the subclass' constructor
- If no explicit call to `super`, then an implicit call to the zero-parameter `super ()` will be made

Custom Exceptions

Often times we need to raise a custom exception

Extend `Exception` **or** `RuntimeException`

Custom Exceptions

What is the difference between extending from `Exception` rather than `RuntimeException`?

Subclass of `Exception` are checked exceptions – must be treated/caught

Subclass of `RuntimeException` are not checkable during compile time

toString

Java classes have a special method defined as `public String toString()`

To understand the purpose of a `toString`, let's see what happens when we don't have one..... (BankAccount.java / Main.java)

The type of the object and the memory address where it is stored are printed.

Classes are collections of many variables, and it is unclear which variables should be printed if you do not explicitly define it. This is the purpose of a `toString()`.

Method Overriding

- Inherited methods from the superclass can be redefined/changed
 - signature stays the same
- **Let's override toString in our code**

Static

- something that belongs to the class itself not any individual instance of the class (objects)
- Circle.java
- accessed by the class name rather than the instance name
- can also have static methods
 - exercise: add a static `printShapeName()` method that prints “Circle”

Exercise - Animal.java

1. What is printed
2. Which toString() runs for each print call
3. What is the final value of count?

Summary

- Exceptions
 - checked (Exception: FileNotFoundException) / unchecked (RuntimeException: AOB, NPE, Arithmetic)
 - try / catch
 - custom exceptions - extend Exception or RuntimeException
- Classes
 - public / private
 - inheritance
 - toString()
 - static

Homework 0 due next Friday Sep 11